QUANSER
INNOVATE·EDUCATE

Lab Procedure
Stepper Motor

Introduction

Ensure the following:

1. You have reviewed the **Application Guide – Stepper**
2. Make sure you have the provided **stepper motor**.
3. Make sure the **Mechatronic Actuators Trainer** is out of its case; it is powered using the provided power supply and connected to the PC.
    a. Turn it on using the Power switch on the side of the device.
    b. Both the USB and Power LED should be green.

This lab introduces the fundamentals of driving stepper motors through two hands-on sections. In Part 1, you'll experiment with wave drive, full-step, and half-step modes to see how coil energizing patterns affect motion. Part 2 focuses on micro stepping, where partial coil activation allows smoother, finer steps. You'll observe how step size, torque, and angular smoothness change with different drive techniques.

Stepper Motor Drive Modes

The concept review mentioned in the application guide introduced the different driving modes for stepper motors. These include wave drive, full step drive, and half step drive. Complete the tables for how to drive the motor in each of these drive modes assuming a clockwise rotation starting with A. Use 0's and 1's for the coil being off and on respectively.

Wave Drive			
A+	A-	B+	B-

Full Step Drive			
A+	A-	B+	B-

Half Step Drive			
A+	A-	B+	B-

1. According to the motor's datasheet (located in the stepper folder), what is the step angle of the motor? Based on this, how many steps are needed to complete a full rotation if we are using wave drive? If we are using full step drive? If we are using half step drive?

2. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**…, and browse to the directory containing the python files and lab procedure for the stepper lab (…/actuators_trainer/1_fundamentals/stepper/hardware/python). Open this directory.

3. Open **stepper.py** and locate the following line:

   ```
   with ActuatorsTrainer(block = 2) as actuators:
   ```

   This line is present in all labs and is used to initialize the device. The parameter **`block = 2`** dictates what block, as printed on the device, you are going to use. You can update this parameter if you connect the motors to a different block.

4. Connect your motor to the block you will be using.
   a. Use the **`Stepper`** connection for the motor power input.

5. Using the tables from above, assign values to the variables **`waveDrive`**, **`fullStepDrive`** and **`halfStepDrive`** at the top of the script.

6. The variable **`stepCounter`** increases every **`nextStep`** steps. Since **`nextStep`** is set to 1, **`stepCounter`** increases once per loop iteration, which is 300 times a second (based on the **`frequency`**). Update the **`cmd = waveDrive[0] * stepperAmplitude`** line so that the command cycles through the values of **`waveDrive`** using **`stepCounter`**.
   NOTE: Make sure to consider that **`stepCounter`** might increase to a value that **`waveDrive`** does not have as an index.

7. Run **stepper.py**, it will run for 30 seconds (**`simulationTime`**). Does your motor turn as expected? If the body of your stepper feels like it is skipping positions, look at your **`waveDrive`** variable or the logic on how you go through the steps.

8. Based on your calculation of steps needed to make a full rotation using **`waveDrive`**, at the top of your script, change the **`simulationTime`** to 1 and the **`frequency`** to the number of steps needed for one rotation. Make a note of where the flat part of your stepper shaft is and run **stepper.py**, did the shaft come back to its original position? Try running it multiple times and see if it always comes back to the same place.

9. How would you make it take 2 turns? What if you wanted the 2 turns in just 1 second? Try it out.

10. Bring your **`simulationTime`** to 10 seconds.

11. Try bringing the **`stepperAmplitude`** down to 0.2 and run **stepper.py** again, try holding the body of the motor and try to stall the motor by holding the shaft, what do you notice? Bring it up to 0.5 now and repeat.

12. Try bringing the **`frequency`** up to 500 and vary the **`stepperAmplitude`** from 0.3 to 0.8. When running **stepper.py**, what do you notice? Feel the body of the motor and try stalling it with your fingers.

13. Reset **`simulationTime`** to 30 seconds, **`frequency`** to 300 Hz, and **`stepperAmplitude`** to 0.3. Repeat steps 6-12 but with **`fullStepDrive`** and then again with **`halfStepDrive`**. Comment out the cmd for the other two you are not using at each time.

14. Go back to using the **`fullStepDrive`**. Without changing the frequency (use 400), make changes to **`nextStep`** so that the motor does one full rotation per second. What logic would you use to do the conversion? What if you wanted 17 steps/second or maybe 270? Use **`stepsPerSecond`** if you need.

15. Modify the code so that the motor changes rotation every 4 seconds, moving between clockwise and counterclockwise rotations. Use the **`cntr4s`** if statement set up in the code.

16. Run **stepper.py**, make sure it runs as expected.

17. Stop, save and close your program.

Microstepping

In this section you will explore microstepping using the stepper motor. Instead of making full steps by fully energizing one coil at a time, they will be turned on with variable current so that the coils form a sine wave to have a smoother motion.

18. Open **microstepping.py** and update the initialization line with the block you will use and connect the stepper motor to the device. This is described above, in step 2 and 3.

19. Note near the top of the file that there is a sine and cosine waves already defined for you based on the **`angularFrequency`** defined before them. These variables will update to the position in the wave at your current timestep.

20. The **Concept Review – Steppers** introduces microstepping. Based on that theory, use the predefined **`sine`** and **`cosine`** variables that update based on the current time, update the **`a_plus`**, **`a_minus`**, **`b_plus`** and **`b_minus`** variables so that the motor rotates clockwise.

21. What is the relationship between the waveforms applied to **`a_plus`** and **`a_minus`**?

22. Run **microstepping.py** and verify that your motor is rotating clockwise as expected.

23. With a **`frequency`** of 500 and an **`angularFrequency`** of 30, as set in the file, how many microsteps are in each half wave? How would you calculate this?

24. The motor should rotate slowly. based on the **`angularFrequency`** of the sine wave, how many seconds should it take to do 1 full rotation? How would you calculate this?

25. Change your code's **`simulationTime`** to the value you found above and run again. Does the motor do 1 full rotation in that time?

26. Bring the **`simulationTime`** back to 30 seconds and change the **`angularFrequency`** to 60. Run **microstepping.py** again. Does the motor rotate faster or slower? Feel the vibration of the motor body when running it.

27. Change the **`frequency`** from 500 to 100 and run again. Does the speed of the motor change? Feel the vibration of the motor body, is it different? Why?

28. Update the **`frequency`** back to 500. To make the motor rotate counterclockwise: Uncomment the line out **`forward = not forward.`** This toggles the **forward** variable every 4 seconds. Replace the line **`pass # do something`** in the **`if forward:`** section with the logic that will cause the motor to rotate in the opposite direction (counterclockwise) when the **`forward`** variable is False. Run **microstepping.py** again.

29. If we wanted at least 8 samples per period of the sine wave, what is the largest angular frequency we can go to (to the nearest 10) if we keep the 500Hz of the python file?

30. Try setting up your angular frequency to this number and run **microstepping.py**. How fast is your motor turning? How does the body of the motor feel when compared to a smaller angular frequency? Try stalling the motor by holding onto the shaft, can you easily do it?

31. Bring the **`stepperAmplitude`** to 0.2 and run again. Try stalling the motor again by holding onto the shaft, how different is it from before?

32. Stop, save and close your program.

33. Power OFF the Mechatronic Actuators Trainer, disconnect any motors and pack them along the power supply and USB cables in the provided case.